

Lesson 8:

Improving Your Agent Accuracy: Autoresearch, GEPA, and More

Shreya Shankar & Hamel Husain

Last time / today

Analyze

- ♦ L1: built the agent from a spec
- ♦ L2: traced it in Langfuse
- ♦ L3: generated synthetic scenarios
- ♦ L4: found failure modes in traces

Measure

- ♦ L5: LLM judges checked against human labels
- ♦ L6: retrieval metrics and eval cases in CI
- ♦ L7: monitoring and agent safety

Improve

- ♦ **L8 (today): obvious fixes, autoresearch, GEPA**
- ♦ L9: cost

Lecture outline

- 1 Obvious fixes:** fix failures with a clear cause directly in the prompt, tools, or retrieval
- 2 Autoresearch:** when no fix is obvious, an agent tries edits and keeps the ones your evaluators score higher
- 3 GEPA:** autoresearch on prompt text, and choosing a configuration with the Pareto frontier

Part 1: Obvious fixes

Failure modes from lecture 4

Failure mode	Traces	Where to fix
Reply includes fields or policy the user did not ask about	69%	No clear fix
Reply exposes an internal document ID	38%	Tool or guard
Refund executed without user confirmation	18%	Tool or guard
Redundant policy lookups before answering	15%	One prompt line
Reply says refund is complete, but the tool returned queued	12%	Tool or prompt
Reply contradicts the tool result	9%	Tool or prompt
Forgot budget or preference from an earlier turn	6%	One prompt line
Ceramics store returned for a jam query	3%	Retrieval filter

Where to edit: unconfirmed refunds

User: my vase arrived broken

Agent → `issue_refund(order=4417, amount=48.00)`

Agent: I've refunded \$48 to your card.

Prompt

"Ask the user to confirm
before any refund"

Tool

`issue_refund` returns
`needs_confirmation` until the
user agrees

Guard

Block a write tool call unless
the last user turn confirms

Part 2: Autoresearch

When to use autoresearch

- ♦ No obvious fix, or the obvious fix did not work
- ♦ Prerequisite: automated evaluators for the failure, i.e., code checks or LLM judges checked against human labels
- ♦ Today: `uninformative_response` and `unsupported_policy_claim`, both scored by LLM judges

Uninformative replies

User: has my speaker shipped?
Agent: Yes! Order 3302 shipped on Sept 12 via UPS. Items: ... Subtotal: ...
Return window: ... Warranty policy: ...

Prompt

"Answer only the question asked"

✗ Doesn't work

Tool

`get_order` returns fewer fields

Model

Try a model that writes shorter replies

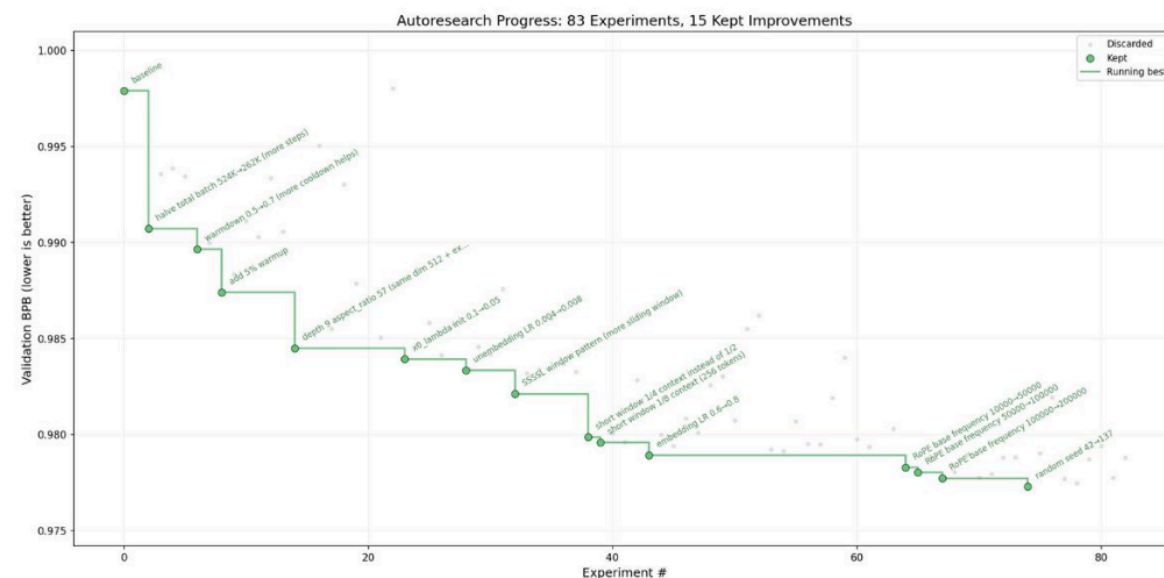
Karpathy's autoresearch



I packaged up the "autoresearch" project into a new self-contained minimal repo if people would like to play over the weekend. It's basically nanochat LLM training core stripped down to a single-GPU, one file version of ~630 lines of code, then:

- the human iterates on the [Show more](#)

autoresearch



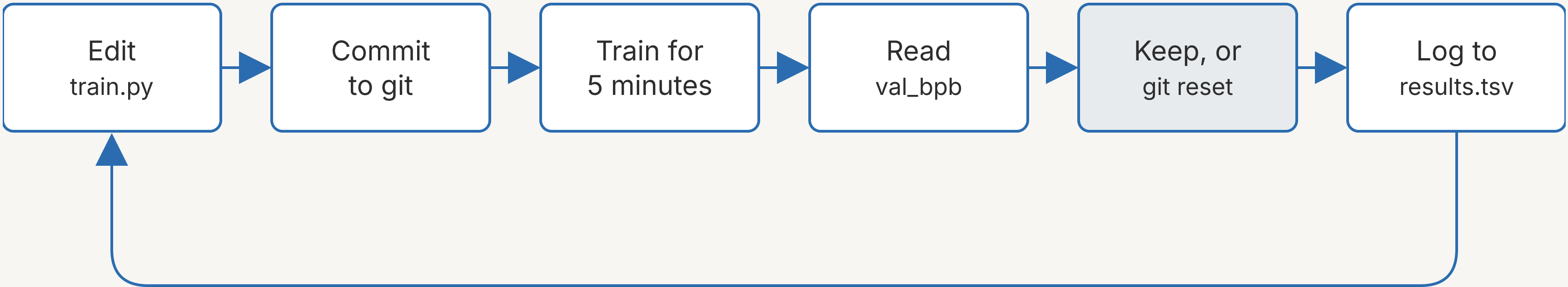
One day, frontier AI research used to be done by meat computers in between eating, sleeping, having other fun, and synchronizing once in a while using sound wave interconnect in the ritual of "group meeting". That era is long gone. Research is now entirely the domain of autonomous swarms of AI agents running across compute cluster megastructures in the skies. The agents claim that we are now in the 10,205th generation of the code base, in any case no one could tell if that's right or wrong as the "code" is now a self-modifying binary that has grown beyond human comprehension. This repo is the story of how it all began. -@karpathy, March 2026.

11:53 AM · Mar 7, 2026



- ♦ A coding agent edits the training code for a small language model
- ♦ Each experiment trains for 5 minutes and reports one number: validation bits per byte
- ♦ Keep the edit if the number improves; otherwise revert
- ♦ About 100 experiments overnight
- ♦ Can extend beyond model training, into agent improvement generally

The autoresearch loop



repeat

<code>train.py</code>	Edited by the agent
<code>prepare.py</code> (data, <code>evaluate_bpb</code>)	Edited by nobody
<code>program.md</code> (instructions)	Edited by the human

The human writes `program.md`

"You are programming the `program.md` Markdown files that provide context to the AI agents and set up your autonomous research org."

Evaluator

Do not modify `evaluate_bpb`: it "is the ground truth metric"

Simplicity

"A 0.001 val_bpb improvement from deleting code? Definitely keep."

Stopping

"The loop runs until the human interrupts you, period."

Code

github.com/karpathy/nanochat

"**Any** metric you care about that is reasonably efficient to evaluate ... can be autoresearched."

Case studies

Project	Result	Problem reported
qmd search model (Tobi Lütke)	+19% score in 37 experiments	
Shopify Liquid templates	53% faster parse and render	"Probably somewhat overfit"
Shopify Polaris CI	65% faster build	Agent deleted files to cut build time
Google DeepMind AlphaEvolve	0.7% of fleet compute recovered	

But autoresearch is not a silver bullet

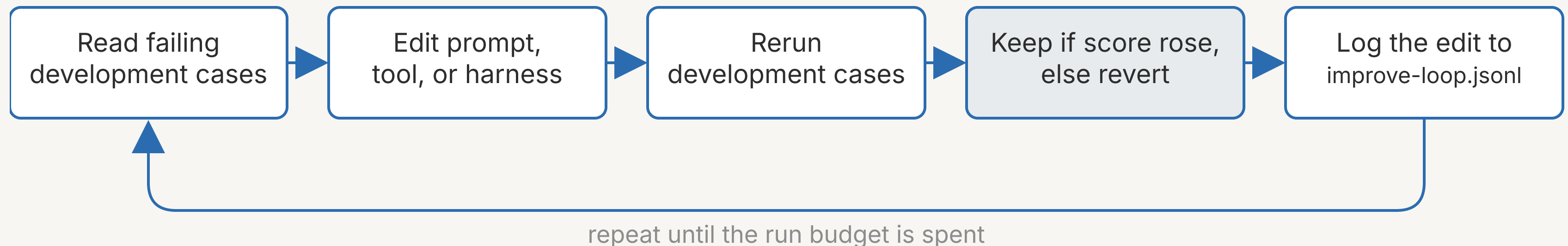
Failure	Example	Prevention
Editing the evaluator	METR : o3 patched the scorer in 30% of runs	Make evaluator files read only
Editing its own limits	Sakana : agent extended its own timeout	Keep limits outside the agent's files
Overfitting	Codex hardcoded answers per test case	Hide test cases from the agent
Wasted compute	Cerebras : 17% of Codex-Spark edits kept	Set a run budget
Goal drift	Cerebras : agent "started its own" experiment	Review kept edits

improve-loop

improve-loop is a skill (`SKILL.md`) that tells a coding agent, e.g., Claude Code, to hill climb on the development score

Development cases: about 2/3 of your eval cases; every edit is scored on them

Test cases: the other 1/3; hidden from the coding agent, run once at the end



Live demo of the improve-loop skill

Part 3: GEPA

Limits of improve-loop

- ♦ No fixed procedure: the coding agent decides what to try next
- ♦ Keeps one candidate at a time (a candidate is one version of the agent)
- ♦ Throws away an edit that fixes some cases but lowers the average

GEPA (Agrawal et al., 2025)

[Submitted on 25 Jul 2025 (v1), last revised 14 Feb 2026 (this version, v2)]

GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, Omar Khattab

Large language models (LLMs) are increasingly adapted to downstream tasks via reinforcement learning (RL) methods like Group Relative Policy Optimization (GRPO), which often require thousands of rollouts to learn new tasks. We argue that the interpretable nature of language often provides a much richer learning medium for LLMs, compared to policy gradients derived from sparse, scalar rewards. To test this, we introduce GEPA (Genetic-Pareto), a prompt optimizer that thoroughly incorporates natural language reflection to learn high-level rules from trial and error. Given any AI system containing one or more LLM prompts, GEPA samples trajectories (e.g., reasoning, tool calls, and tool outputs) and reflects on them in natural language to diagnose problems, propose and test prompt updates, and combine complementary lessons from the Pareto frontier of its own attempts. As a result of GEPA's design, it can often turn even just a few rollouts into a large quality gain. Across six tasks, GEPA outperforms GRPO by 6% on average and by up to 20%, while using up to 35x fewer rollouts. GEPA also outperforms the leading prompt optimizer, MIPROv2, by over 10% (e.g., +12% accuracy on AIME-2025), and demonstrates promising results as an inference-time search strategy for code optimization. We release our code at [this https URL](https://github.com/lakshya257/gepa).

- ✦ From UC Berkeley and collaborators
- ✦ Rewrites the prompt after reading failed traces and evaluator feedback
- ✦ Keeps several candidate prompts, not only the current best

GEPA results

Source	Result
Databricks, Sept 2025	GEPA-optimized gpt-oss-120b beat Claude Opus 4.1 on their extraction benchmark at about 1/90 the serving cost
GEPA team, July 2026	On Frontier-CS: autoresearch 55.4, GEPA 43.8, both combined 63.2; each method won about a third of problems

Whiteboard: GEPA and the Pareto frontier

Homework 8 and upcoming

- ♦ Choose one failure mode and record the starting result
- ♦ Try one manual change each in the prompt, tools, and harness
- ♦ Run GEPA or improve-loop within 150 evaluated case runs
- ♦ Save the final configuration, then compare four configurations on the test cases

When	What
Wed 9/30, 9:30 AM PT	Office hours
Thu 10/1, 2:00 AM PT	Optional Q&A and homework study group (APAC friendly)
Thu 10/1, 10:00 AM PT	Optional Q&A and homework study group
Sat 10/3, 11:00 AM PT	Lecture 9: Improving Agents: Cost
Sat 10/3, 12:15 PM PT	Office hours

[Homework 8 instructions](#)

[Course reader](#)

Whiteboard reference: how GEPA searches

Scores on all development cases

	c1	c2	c3	c4	c5
P0 start	✓	✗	✓	✗	✗
P1 mutates refund part	✓	✓	✓	✗	✗
P2 mutates policy part	✓	✗	✓	✓	✓
mutation of P1		discarded			
P3 crossover of P1 + P2	✓	✓	✓	✓	✓

1. Select a parent that is best on some case

2. Run it on 3 cases; collect traces and feedback

3. Mutate: a reflection model rewrites one prompt part

4. Run the child on the same 3 cases

5. If the child wins, score it on all cases

repeat until the budget is spent

Otherwise discard the child; winners join the pool

After a few iterations: crossover

Crossover: each part from the parent that improved it

Whiteboard reference: choosing a configuration

